Theoretical Guide
py.Sandu

Élvis Miranda, Leonardo Krauss & Rafael Alencar

1 C++

1.1 System optimization

For faster IO, we can use the following code:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
cout.tie(nullptr);
```

This code disables the synchronization between C++ streams and C streams, which can significantly improve the performance of input and output operations. Additionally, it unties the input and output streams, allowing them to operate independently, further enhancing the speed of IO operations.

1.2 Containers

- **std::vector:** A dynamic array. Most methods give O(1) time complexity for access and O(N) for insertion/deletion (except at the end, which is O(1)).
- **std::list:** A doubly linked list. Most methods give O(1) time complexity for insertion/deletion (given an iterator) and O(N) for access.
- **std::deque:** A double-ended queue. Most methods give O(1) time complexity for access and O(1) for insertion/deletion at both ends, but O(N) for insertion/deletion in the middle.
- **std::stack:** A container adapter that gives the functionality of a stack (LIFO). Most methods give O(1) time complexity for push, pop, and top operations.
- **std::queue:** A container adapter that gives the functionality of a queue (FIFO). Most methods give O(1) time complexity for push, pop, and front/back operations.
- **std::priority_queue:** A container adapter that gives the functionality of a priority queue. Most methods give O(log N) time complexity for push and pop operations, and O(1) for top operation.
- **std::set:** A sorted associative container that contains unique elements. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::map:** A sorted associative container that contains key-value pairs with unique keys. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::unordered_set:** An unordered associative container that contains unique elements. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.
- **std::unordered_map:** An unordered associative container that contains key-value pairs with unique keys. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.

1.3 Algorithms

- **std::sort:** The `std::sort` function in C++ is typically implemented using a hybrid sorting algorithm called Introsort, which combines Quick Sort, Heap Sort, and Insertion Sort. It has an average time complexity of O(N log N).
- **std::lower_bound and std::upper_bound:** These functions are used to find the lower and upper bounds of a value in a sorted range. They have a time complexity of O(log N) since they use binary search.
- **std::next_permutation:** This function generates the next lexicographical permutation of a sequence. It is useful for generating all permutations of a sequence in sorted order. The time complexity is O(N) for each call, where N is the length of the sequence.

2 Number Theory

- **Euclide's Lemma:** $\forall a, b, p \in \mathbb{Z}, p$ is prime, $p|ab \Rightarrow p|a$ or $p|b$.
- **Bézout's identity:** $\forall x, y \in \mathbb{Z}^*, \exists a, b \in \mathbb{Z} : ax + by = \gcd(a, b)$.

3 Constants

3.1 Time and memory limits

- **1 second limit:** 1 second is usually enough for a program to perform around 10^8 operations for O(N) algorithms and around 10^6 operations for O(N log N) algorithms.

- **256Mb limit:** 256 megabytes is usually enough to store around 10^7 integers (since each integer typically takes 4 bytes).

3.2 Mathematical

- **Pi (π)**: Usually defined as acos(-1.0), approximately equal to 3.14159. In C++ 20, it is available as `std::numbers::pi`.

- **Euler (e)**: Usually defined as the base of the natural logarithm, approximately equal to 2.71828. In C++ 20, it is available as `std::numbers::e`.

- **Golden Ratio (ϕ)**: Usually defined as (1 + sqrt(5)) / 2, approximately equal to 1.61803. In C++ 20, it is available as `std::numbers::phi`.

3.3 Data type limits

- **Integer limits**: The typical 32 bit signed integer bounds are ($\approx \pm 2 \times 10^9$) and is obtained by using the `INT_MAX` and `INT_MIN` constants, while the 64 bit signed integer bounds are ($\approx \pm 9 \times 10^{18}$) and is obtained by using the `LLONG_MAX` and `LLONG_MIN` constants.

- **Floating-point limits**: The typical 32 bit floating-point bounds are approximately $\pm 3.4 \times 10^{38}$ and is obtained by using the `FLT_MAX` and `FLT_MIN` constants, while the 64 bit floating-point bounds are approximately $\pm 1.7 \times 10^{308}$ and is obtained by using the `DBL_MAX` and `DBL_MIN` constants.

4 Math

5 Notes

6 Identities

7 Geometry

8 Bitwise

9 Counting Problems

10 Progressions

A progression is a sequence of numbers ordered according to a specific, logical rule or formula, where each term is derived from the previous one.

10.1 Odds

General Term: $(2n-1)^2$

Sum: $\dfrac{n(4n^2-1)}{3}$

Every odd number can be represented as $2n+1$

10.2 Geometric Progression

General Term: $a_1 \times q^{n-1}$

Sum: $\dfrac{a_1(q^n-1)}{q-1}$

Infinite Sum:

$$-1 < q < 1$$

$$\frac{a_1}{1-q}$$

10.3 Arithmetic Progression

General Term: $a_1 + (n-1)r$

Sum: $\frac{(a_1 + a_n)n}{2}$